

Universidad Politécnica de Madrid
Escuela Técnica Superior de Ingenieros de Telecomunicación

**Clasificación Automática de
Afirmaciones Políticas
mediante Procesamiento del Lenguaje Natural**

Asignatura ABID — Kaggle Challenge II

Grupo 1

Marla González Escobar
Belén María Iniesta Tejera
Silvia Rodríguez Hernández
Rosali Rodriguez Ureña
Marilyn Salazar Martínez
Iván Omar Angeles Martínez

Mayo de 2026

Índice

1. Introducción	2
2. Descripción del problema	2
3. Descripción del conjunto de datos	2
3.1. Preprocesamiento y generación de variables	4
4. Descripción de tareas comunes	4
5. Evaluación	5
5.1. Modelos evaluados	5
5.1.1. Modelos lineales	5
5.1.2. Modelos <i>ensemble</i>	6
5.1.3. Transformers	6
5.1.4. Estrategias de ensamble	7
5.2. Resultados	7
6. Conclusiones	8

1. Introducción

La detección automática de afirmaciones falsas es una tarea cada vez más relevante dentro del Procesamiento del Lenguaje Natural, especialmente en contextos políticos, donde la desinformación puede influir de forma significativa en la opinión pública. Este proyecto aborda un problema de clasificación binaria: predecir si una afirmación política es verdadera o falsa a partir de su contenido textual e información contextual como el emisor, su afiliación política o el tema tratado.

Para resolver el problema se evaluaron distintos enfoques de aprendizaje automático, desde modelos clásicos hasta modelos más avanzados basados en *embeddings* semánticos, técnicas de *ensemble* y *boosting*. El mejor resultado del equipo alcanzó un **Macro F1 de 0,69638** en el *leaderboard* público de Kaggle [1], mediante una estrategia híbrida de *distillation* desde un *ensemble* de Mistral-7B [2] y DeBERTa-v3-large [3].

Esta memoria resume el análisis del conjunto de datos, el preprocesamiento realizado, los modelos desarrollados, la metodología de evaluación y las principales conclusiones obtenidas.

2. Descripción del problema

El objetivo del proyecto consiste en desarrollar un sistema capaz de clasificar afirmaciones políticas como verdaderas o falsas utilizando técnicas de Procesamiento del Lenguaje Natural y aprendizaje automático. Se trata de un problema de clasificación binaria, donde tenemos que identificar si una afirmación es *fake news* o no.

El *input* principal es el texto de la afirmación, pero también se cuenta con metadatos como tema, *speaker*, ocupación, estado y afiliación política, que pueden aportar señal adicional al modelo.

El reto central es que la veracidad no siempre se puede inferir solo del texto: dos afirmaciones pueden ser lingüísticamente similares y pertenecer a clases opuestas. A eso se suma el *class imbalance* del *dataset*, razón por la que se usa **Macro F1** como métrica principal, ya que permite evaluar el rendimiento de forma equilibrada entre ambas clases.

3. Descripción del conjunto de datos

El conjunto de datos utilizado [4] contiene afirmaciones políticas acompañadas de información contextual relacionada con cada una de ellas. Cada registro incluye el texto de la afirmación, recogido en la variable `statement`, junto con distintos metadatos como el tema tratado (`subject`), el hablante (`speaker`), su ocupación (`speaker_job`), el estado (`state_info`) y la afiliación política (`party_affiliation`). La variable objetivo

es `label`, donde el valor 1 representa una afirmación falsa y el valor 0 una afirmación verdadera.

Cuadro 1: Variables del *dataset*.

Variable	Tipo	Descripción y Pre-Procesamiento
<code>id</code>	Identificador	Solo para trazabilidad; excluido del modelo.
<code>label</code>	Target binario	0 = verdadero, 1 = falso (es fake news).
<code>statement</code>	Texto	La afirmación política. se utilizaron análisis lexicográficos para procesarla.
<code>subject</code>	Categórica	Tema(s) de la afirmación; reducción de dimensiones
<code>speaker</code>	Categórica	Persona que realiza la afirmación. Limpieza de datos.
<code>speaker_job</code>	Categórica	Cargo del hablante (27,7 % de valores ausentes).
<code>state_info</code>	Categórica	Estado geográfico de EE. UU. reducida.
<code>party_affiliation</code>	Categórica	Afiliación política del hablante.

Durante el análisis exploratorio inicial se observó que el conjunto de entrenamiento contiene **8.950 muestras** y el de test **3.836**, con un cierto desbalanceo entre clases. Aproximadamente el 65 % de las muestras pertenecen a la clase verdadera, mientras que el 35 % corresponden a afirmaciones falsas. Este desequilibrio se tuvo en cuenta en la elección de la métrica de evaluación, dando prioridad a Macro F1, ya que permite valorar de forma más equilibrada el rendimiento en ambas clases.

También se detectaron valores nulos en las variables `speaker_job` (27,7 % de valores ausentes) y `state_info` (~22 %). En lugar de eliminar estos registros, los valores faltantes se sustituyeron por la categoría **unknown**, conservando así todas las muestras disponibles y permitiendo que los modelos pudieran tratar la ausencia de información como una categoría más.

Otro aspecto relevante del *dataset* es la alta cardinalidad de algunas variables categóricas: `speaker` (2.634 valores únicos) y `speaker_job` (1.018 únicos), donde el 84,6 % de los cargos aparece menos de cinco veces. Para reducir ruido y mejorar la generalización, las categorías poco frecuentes se agruparon bajo la etiqueta **other**.

Otro aspecto relevante fue la calidad de la columna `statement`, que presentó un alto grado de corrupción: caracteres Unicode mal codificados, fragmentos JSON embebidos en el texto y secciones del archivo originalmente en formato TSV en lugar de CSV, lo que provocaba desplazamientos de columnas. Fue necesario implementar un proceso de limpieza específico para recuperar correctamente el texto de cada afirmación antes de cualquier representación textual.

Por último, la columna `subject` podía contener varios temas separados por comas para una misma afirmación. Por ello, se realizó una limpieza de esta variable y se generaron características auxiliares, como una versión normalizada del texto y el número de temas asociados a cada afirmación.

3.1. Preprocesamiento y generación de variables

Se realizó una limpieza general de las variables categóricas y textuales: normalización a minúsculas, sustitución de nulos en `speaker_job` y `state_info` por `unknown`, y agrupación de categorías poco frecuentes bajo la etiqueta `other`. La variable `subject`, que podía contener varios temas por afirmación, se procesó con `MultiLabelBinarizer` [6].

Para los modelos clásicos basados en TF-IDF [6] se aplicó preprocesamiento textual agresivo sobre `statement`: eliminación de puntuación, normalización de contracciones, tokenización, eliminación de *stopwords* y lematización con NLTK [12], conservando negaciones relevantes. La representación combinó TF-IDF con *n*-gramas de palabras y caracteres junto con *embeddings* semánticos de Sentence Transformers [11] (`all-MiniLM-L6-v2`, `all-mpnet-base-v2`).

Se generaron también variables derivadas del comportamiento histórico de los hablantes: tasas de veracidad por hablante, partido y tema. Para evitar *leakage*, estas estadísticas se calcularon con estrategia *out-of-fold* [6] dentro de la validación cruzada.

En los modelos Transformer [14] no se aplicó limpieza agresiva; en su lugar se construyó un texto enriquecido (`model_text`) que combina la afirmación con la *metadata* contextual en formato natural, siguiendo el esquema:

```
Speaker: <speaker>| Party: <party>| Job: <job>| State: <state>| Topic:
<subject>[SEP] Claim: <statement>
```

donde [SEP] es el token separador estándar del *tokenizer*. Este enfoque permite al modelo tratar los metadatos como tokens semánticos y aprender automáticamente sus interacciones con el contenido del `statement`, sin necesidad de codificación *one-hot* ni *embeddings* categóricos explícitos.

4. Descripción de tareas comunes

El proyecto se desarrolló de forma colaborativa: cada integrante trabajó sobre etapas o modelos distintos, mientras que las decisiones de preprocesamiento, evaluación y comparación de resultados se tomaron de forma conjunta. Se utilizó GitHub [5] como repositorio del equipo (<https://github.com/DataOrDie/truth-classifier-nlp>) y Kaggle [1] para el entrenamiento de modelos, generación y envío de *submissions*. La búsqueda de hiperparámetros se realizó con `RandomizedSearchCV` [6]; en los modelos

Transformer se ajustaron adicionalmente *learning rate*, *batch size*, *dropout*, número de épocas, umbral de decisión y *warmup*.

Herramientas y tecnologías

Cuadro 2: Herramientas utilizadas en el proyecto.

Categoría	Herramienta	Uso principal
Lenguaje	Python 3.x [7]	Todo el <i>pipeline</i> .
ML clásico	scikit-learn [6]	LR, RFC, KFold, búsqueda de H
Gradient Boosting	LightGBM / CatBoost / XGBoost [8, 9, 10]	Modelos <i>boosted</i> y <i>stacking</i> .
<i>Embeddings</i>	sentence-transformers [11]	all-MiniLM-L6-v2, all-mpnet-b
NLP / NER	NLTK + spaCy [12, 13]	<i>Stemming</i> , lematización, entidad
Transformers	HuggingFace + PEFT + bitsandbytes [14, 15, 16]	DeBERTa [3], Mistral-7B LoRA [
<i>Tracking</i>	Weights & Biases [17]	Métricas, curvas, artefactos.
Otros	joblib / GitHub / Kaggle [18, 5, 1]	Serialización, versiones, <i>submissio</i>

5. Evaluación

5.1. Modelos evaluados

Con el objetivo de comparar distintos enfoques para la detección de afirmaciones falsas, se evaluaron modelos de diferente complejidad, desde algoritmos lineales clásicos hasta modelos *ensemble* y arquitecturas Transformer. La comparación permitió analizar tanto el rendimiento predictivo como la capacidad de generalización de cada técnica sobre el *dataset*.

Los modelos fueron entrenados utilizando la misma estrategia de validación y conjuntos de datos procesados, adaptando únicamente la representación de las variables según las necesidades de cada algoritmo. Las métricas principales empleadas para la comparación fueron **Macro F1** y **ROC-AUC**, ya que el *dataset* presentaba desbalanceo entre clases.

5.1.1. Modelos lineales

Como *baseline* se evaluaron Regresión Logística y Linear SVM con representaciones TF-IDF (*n*-gramas de palabras y caracteres) [6], regularización L1/L2 y `class_weight` balanceado. La Regresión Logística ofreció buena interpretabilidad y resultados competitivos, aunque su carácter lineal le impide capturar interacciones complejas entre texto y *metadata*.

5.1.2. Modelos *ensemble*

Se evaluaron Random Forest [6], XGBoost [10], LightGBM [8] y CatBoost [9], combinados con *embeddings* semánticos [11] y variables derivadas del historial del hablante. Estos modelos capturaron relaciones no lineales que los modelos lineales no podían representar; CatBoost destacó por su manejo nativo de variables categóricas y el control de *leakage* mediante *ordered boosting*.

Finalmente, se implementó *stacking* con meta-clasificador Logistic Regression [6], aprovechando la complementariedad entre los modelos base y mejorando tanto Macro F1 como ROC-AUC respecto a los modelos individuales.

5.1.3. Transformers

La última familia de modelos evaluados estuvo basada en arquitecturas Transformer preentrenadas [14]. Estos modelos permiten capturar relaciones contextuales y semánticas más complejas que los enfoques basados únicamente en TF-IDF o *embeddings* estáticos.

Para los Transformers se utilizó texto enriquecido con *metadata* contextual, incorporando información como el hablante, el tema y la afiliación política directamente en la entrada textual del modelo. Además, se ajustaron hiperparámetros como *learning rate*, *batch size*, *dropout* y número de épocas para optimizar el rendimiento. Para los experimentos de *fine-tuning* eficiente se emplearon las librerías PEFT y bitsandbytes [15, 16].

Los modelos Transformer mostraron mejores resultados que los modelos clásicos cuando se combinaron con información contextual. DeBERTa-v3-base con *late fusion* en el *stacking* alcanzó Macro F1 = 0,647 y ROC-AUC = 0,711 en *holdout*.

Para escalar la capacidad del *encoder* se entrenó **DeBERTa-v3-large** [3] (435 M parámetros, atención *disentangled* y preentrenamiento estilo ELECTRA), el mejor modelo *encoder* del equipo con Macro F1 = 0,667 en el *leaderboard* de Kaggle. Para que cupiera en GPU T4 se usó `batch_size=8` con `gradient_accumulation_steps=2` y *gradient checkpointing*. Se aplicó *class weight* (1,85; 1,0) para combatir el desbalance, *label smoothing* de 0,05, *cosine LR schedule* con *warmup* del 10 % y *learning rate* = 1e-5.

Para superar ese techo se probó LoRA (*Low-Rank Adaptation*) [15] sobre `mistralai/Mistral-7B-v` cuantizado en 4 bits con bitsandbytes (NF4) [16]. LoRA introduce matrices de rango bajo en las capas de atención y entrena únicamente esos ~10–50 MB adicionales en lugar de los ~14 GB del modelo completo, lo que hace viable el entrenamiento en GPU T4. Con *k-fold* de 3 y *learning rate* = 5e-5, el modelo alcanzó Macro F1 = **0,676** en Kaggle (0,670 en *holdout* interno), convirtiéndose en el mejor modelo individual del proyecto. Un hallazgo clave fue que añadir los GBDT al *stacking* en este punto *empeoró* el resultado: el coeficiente del transformer en el meta-clasificador ascendió a 2,75 frente

a 0,08–0,50 de los modelos de árboles, evidencia de que estos ya no aportaban señal útil. El transformer con *threshold sweep* directo sobre las predicciones OOF superó al *stacking* completo en más de 0,05 puntos de Macro F1.

5.1.4. Estrategias de ensamble

Una vez entrenados los modelos individuales más fuertes (Mistral-7B y DeBERTa-v3-large), se exploraron tres estrategias de combinación que produjeron los mayores saltos de rendimiento del proyecto.

Voting con confianza Mistral $\times$ DeBERTa-large. Partiendo de las predicciones binarias de Mistral-7B, se sobrescriben los casos en que DeBERTa-large discrepa con probabilidad $\geq 0,70$ o $\leq 0,30$. Esta estrategia explota la diversidad arquitectónica (LLM *decoder-only* frente a Transformer *encoder* bidireccional): de los 3.836 ejemplos del test se modificaron 175 predicciones (4,6 %), obteniendo Macro F1 = 0,687 en Kaggle.

Voting *speaker-aware*. Refinamiento del voting anterior con umbrales adaptados al perfil del orador: en *speakers* donde DeBERTa-large es históricamente fiable (precisión OOF > 80 %) se relaja el umbral a 0,50/0,50 (más *flips*); en *speakers* poco fiables (< 60 %) se endurece a 0,85/0,15 (menos *flips*, mayor confianza en Mistral). Resultado: Macro F1 = 0,689 en Kaggle.

Distillation desde el voting ensemble. Las predicciones del voting Mistral $\times$ DeBERTa-large (F1 = 0,687) se emplearon como *pseudo-labels* del test. Se reentrenó un nuevo DeBERTa-v3-large sobre las 8.950 muestras de entrenamiento más las 3.836 *pseudo-labels* con peso 0,7. El modelo destilado aprendió patrones que ningún modelo individual capturaba, ya que su “maestro” (el voting) era más preciso que cualquier componente por separado, obteniendo el **mejor resultado del equipo: Macro F1 = 0,696** en Kaggle.

5.2. Resultados

La evaluación de los modelos se realizó utilizando validación cruzada estratificada de cinco *folds* y un conjunto *holdout* reservado para la evaluación final [6]. Debido al desbalanceo presente en el *dataset*, las métricas principales utilizadas fueron Macro F1 y ROC-AUC, ya que permiten evaluar de forma más equilibrada el rendimiento sobre ambas clases.

A lo largo del proyecto se observó una mejora progresiva del rendimiento al incorporar representaciones textuales más avanzadas, variables contextuales y técnicas de

combinación de modelos. Los modelos lineales basados en TF-IDF proporcionaron *base-lines* sólidos, mientras que los modelos *ensemble* y Transformer consiguieron capturar relaciones más complejas entre el texto y la *metadata* política.

La Tabla 3 muestra la progresión del equipo en el *leaderboard* público de Kaggle, incluyendo las estrategias de ensamble.

Cuadro 3: Progresión en el *leaderboard* público de Kaggle (Macro F1).

Modelo / Estrategia	Macro F1
TF-IDF + Logistic Regression (<i>baseline</i>)	0,623
LightGBM + <i>features</i> ricas	0,603
ELECTRA-large con <i>metadata</i>	0,639
RoBERTa-large	0,642
DeBERTa-v3-base (<i>multi-seed</i>)	0,658
DeBERTa-v3-large	0,667
Mistral-7B + LoRA (3 <i>folds</i>)	0,676
Voting Mistral × DeBERTa-large	0,687
Voting <i>speaker-aware</i>	0,689
Distillation desde voting	0,696

Uno de los factores que más contribuyó a la mejora del rendimiento fue la incorporación de *embeddings* semánticos [11] y variables contextuales relacionadas con el hablante y el tema de la afirmación. También se comprobó que algunas variables históricas, aunque muy predictivas, podían introducir problemas de sobreajuste si no se calculaban correctamente mediante estrategias *out-of-fold*.

Los modelos Transformer [14, 3] mejoraron notablemente al incorporar *metadata* contextual directamente en el texto de entrada. Hasta DeBERTa-v3-base los mejores resultados se obtenían mediante *late fusion* con modelos *ensemble*; al escalar a Mistral-7B con LoRA [15, 2], el patrón se invirtió: el LLM capturaba señal tan diferente a la de los GBDT que añadirlos introducía ruido. Finalmente, las estrategias de voting y *distillation* recuperaron la complementariedad al aprovechar la diversidad arquitectónica real (LLM *decoder-only* frente a *encoder* bidireccional).

6. Conclusiones

En este proyecto se abordó un problema de clasificación de afirmaciones políticas [4] mediante técnicas de procesamiento del lenguaje natural y aprendizaje automático. A lo largo del desarrollo se evaluaron distintos enfoques, desde modelos lineales clásicos hasta arquitecturas Transformer [14, 3] y estrategias *ensemble* avanzadas [8, 9, 10].

Los resultados obtenidos muestran que la información contextual asociada a las afirmaciones, como el hablante, el tema o la afiliación política, tuvo un impacto muy

relevante en el rendimiento de los modelos. La combinación de esta *metadata* con representaciones textuales avanzadas [11] permitió mejorar significativamente las métricas de clasificación.

Los modelos lineales basados en TF-IDF [6] ofrecieron *baselines* sólidos y eficientes, mientras que los modelos *ensemble* [8, 9, 10] lograron capturar relaciones no lineales entre variables y mejorar la capacidad predictiva. Por su parte, los modelos Transformer [14, 3] consiguieron representar mejor el contexto semántico del lenguaje natural, especialmente al incorporar información contextual directamente en el texto de entrada.

También se observó la importancia de controlar el sobreajuste y evitar *leakage* en variables relacionadas con el historial de veracidad de los hablantes. El uso de validación cruzada estratificada [6] y estrategias *out-of-fold* permitió reducir este problema y obtener estimaciones más realistas del rendimiento.

En la fase de modelos individuales, el *stacking* con GBDT resultó contraproducente cuando el transformer dominaba el *ensemble*: aplicar directamente un *threshold sweep* sobre las predicciones OOF del transformer mejoró el Macro F1 en más de 0,05 puntos. De igual modo, los *stacks* con muchos modelos clásicos enriquecidos mejoraban la validación cruzada pero empeoraban en el *leaderboard*, evidenciando sobreajuste a los *folds*.

El mejor resultado del proyecto se alcanzó mediante *distillation* desde un voting ensemble: las predicciones combinadas de Mistral-7B [2] y DeBERTa-v3-large [3] se usaron como *pseudo-labels* del test para reentrenar un nuevo DeBERTa-v3-large, logrando **Macro F1 = 0,696** en el *leaderboard* público de Kaggle. El factor determinante fue la diversidad arquitectónica real entre ambos modelos (LLM *decoder-only* frente a *encoder* bidireccional): el *pseudo-labeling* desde DeBERTa-large sobre sí mismo, en cambio, no aportó señal nueva (correlación de 0,98 con el original), y añadir RoBERTa-large o ELECTRA-large al voting empeoró el resultado, evidenciando que más modelos no equivale a mejor *ensemble*.

En conclusión, el proyecto permitió comparar técnicas de clasificación textual desde modelos lineales hasta LLM de 7B parámetros, comprobando cómo la integración de información contextual, el ajuste cuidadoso de hiperparámetros, la elección correcta de la estrategia de *ensemble* y, finalmente, la *distillation* desde un *ensemble* de modelos arquitectónicamente diversos son determinantes para el rendimiento en tareas de detección de desinformación política.

Referencias

- [1] Kaggle. Kaggle — your machine learning and data science community, 2025. URL <https://www.kaggle.com/>. Accedido: mayo 2025.
- [2] Mistral AI. Mistral-7B-v0.1 — hugging face model card, 2023. URL <https://huggingface.co/mistralai/Mistral-7B-v0.1>. Accedido: mayo 2025.
- [3] Microsoft. DeBERTa-v3-base — hugging face model card, 2023. URL <https://huggingface.co/microsoft/deberta-v3-base>. Accedido: mayo 2025.
- [4] William Yang Wang. “Liar, Liar Pants on Fire”: A new benchmark dataset for fake news detection. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL 2017)*, 2017. URL <https://arxiv.org/abs/1705.00648>.
- [5] GitHub, Inc. GitHub — where the world builds software, 2025. URL <https://github.com/>. Accedido: mayo 2025.
- [6] scikit-learn developers. scikit-learn: Machine learning in Python — user guide, 2024. URL https://scikit-learn.org/stable/user_guide.html. Accedido: mayo 2025.
- [7] Python Software Foundation. Python 3 reference manual, 2024. URL <https://docs.python.org/3/>. Accedido: mayo 2025.
- [8] Microsoft / LightGBM Contributors. LightGBM documentation, 2024. URL <https://lightgbm.readthedocs.io/en/stable/>. Accedido: mayo 2025.
- [9] Yandex / CatBoost Contributors. CatBoost documentation, 2024. URL <https://catboost.ai/docs/>. Accedido: mayo 2025.
- [10] XGBoost Contributors. XGBoost documentation, 2024. URL <https://xgboost.readthedocs.io/en/stable/>. Accedido: mayo 2025.
- [11] Nils Reimers and Iryna Gurevych. Sentence-transformers documentation, 2024. URL <https://www.sbert.net/>. Accedido: mayo 2025.
- [12] NLTK Project. NLTK: Natural language toolkit documentation, 2024. URL <https://www.nltk.org/>. Accedido: mayo 2025.
- [13] Explosion AI. spaCy — industrial-strength natural language processing, 2024. URL <https://spacy.io/>. Accedido: mayo 2025.
- [14] Hugging Face. Transformers documentation, 2024. URL <https://huggingface.co/docs/transformers/>. Accedido: mayo 2025.

- [15] Hugging Face. PEFT: State-of-the-art parameter-efficient fine-tuning methods, 2024. URL <https://huggingface.co/docs/peft/>. Accedido: mayo 2025.
- [16] Hugging Face. bitsandbytes documentation, 2024. URL <https://huggingface.co/docs/bitsandbytes/>. Accedido: mayo 2025.
- [17] Weights & Biases. Weights & Biases documentation, 2024. URL <https://docs.wandb.ai/>. Accedido: mayo 2025.
- [18] joblib Contributors. joblib: Running python functions as pipeline jobs, 2024. URL <https://joblib.readthedocs.io/>. Accedido: mayo 2025.